00660121 - Medical Diagnostics Final Project: convolutional and transformer models for RNA-protein binding and nanopore basecalling

TA in charge: Mattan Hoory*
Faculty of Biotechnology and Food Engineering
Technion - Israel Institute of Technology

Spring 2026

Submission Due: 09.09.26

Last updated: 20.08.26

Table of contents

*Thanks to Prof. Roee Amit and the Technion Faculty of Biotechnology & Food Engineering for their support, and to the CS 236605 Deep Learning course and the authors of the original assignments and works for inspiration and selected materials.

1 Aim of the assignment

Convolutional Neural Networks (CNNs) and Transformers are two landmark deep learning architectures. The goal of this project is to (1) understand how they work in practice, (2) apply them to sequence data to solve problems in computational biology, and (3) learn about modern methodologies for implementing and training them.

We start with sequence classification: a CNN that predicts whether an RNA-binding protein (RBP) binds a given sequence. We then use the trained model to find the sequence motifs it learned (Task 2.1). Next we move to sequence-to-sequence modelling with the architecture behind modern language models: a Transformer that reads a noisy nanopore current signal and calls the DNA sequence behind it, a task known as *basecalling* (Task 2.2). The Transformer's core idea, self-attention, is the same one that powers language models, and we will see why it succeeds where a CNN cannot.

Both tasks build directly on how you built a network from layers in HW2. What is new is the architecture in each case, and what you can learn by looking inside a trained model to explain its behavior (known as explainability or XAI).

In each task we provide a "skeleton" (Jupyter notebook) to help you get started and focus on the main concepts instead of the code structure. Some parts are implemented and can be used as is, while others are left for you to complete and are marked with `# TODO-n` comments.

1.1 Guidelines

Work in pairs. One submission per pair. **Late submission:** 10% penalty per day. **Submission:** .ZIP file through Moodle, see Section 3.

- *Parts* are marked *A.*, *B.*, …. You should complete them in the provided Jupyter notebooks. Either by filling in the TODOs or by writing your own code. The completed notebooks should be included in your submission. You will not be evaluated on the code itself, but on the results you obtain and the report you write.
  - Each notebook contains its own instructions, so read it alongside this document. You can edit and run the notebooks in VSCode, Jupyter Notebook, Google

Colab, or Kaggle as described in HW0.

- *Questions/Plots* are marked *Q1, Q2, ….* These require you to answer the questions or use code to create the plots, and then include them in your report.

1.2 Resources

All the code lives in the course repository. Get it with either:

- **Download all the code (ZIP)**, then unzip it and open the notebooks in Jupyter or VSCode.
- or clone it with `git clone https://github.com/mathoory/00660121-medical-diagnostics` (see HW0).

The ZIP contains the whole course repository, which is the simplest way to always get the current version. If something is updated you can re-download it and copy over just the notebook you need.

Inside, the notebooks for this project are under `homeworks/04 Final Project/src/`:

Notebook	What you will do in it
`task-1/task-1.ipynb`	Task 1 - train a CNN to recognise RNA sequences that bind the AGO2 protein, then look at what it learned.
`task-2/task-2.ipynb`	Task 2 - build a Transformer that reads raw nanopore signal and calls the DNA bases behind it.

1.3 Running on a GPU (read this before you start)

Both tasks in this project need a GPU. Deep learning is millions of matrix multiplications, which a GPU performs in parallel and finishes roughly 10-30x faster than a laptop CPU.

On a CPU, one training epoch of Task 1 can take an hour instead of a few minutes.

You have free GPU access on Google Colab and on Kaggle Kernels, and both give you a limited number of GPU hours per week, which is enough for this project if you plan your runs. See HW0 for how to get started on either.

> Video walkthrough (from last year - different project, same method, watch from 1:23): enabling a GPU on Google Colab

How to not waste your quota. The GPU clock runs whether or not your code is correct, and most of the time you lose is spent on runs that crash three minutes in, or that you did not need in the first place.

1. **Debug on CPU first, on a deliberately tiny problem.** In Task 1, load only a few hundred sequences and train for one epoch. In Task 2, drop the number of training `steps` to a few dozen. You are not looking for good accuracy here, only for code that runs end to end without errors.
2. **Only then switch the runtime to GPU** and scale up to the full data and a realistic number of epochs.
3. **Decide what each run is testing before you launch it.** Changing three hyperparameters at once tells you nothing about which one mattered.
4. **Save your results as you go** (metrics, plots), so a disconnected session does not cost you a rerun. Colab and Kaggle both disconnect idle sessions.

2 Tasks

2.1 Task 1 - CNN for RNA-protein binding sites

2.1.1 Introduction

RNA-binding proteins (RBPs) regulate RNA after transcription. Cross-linking and immunoprecipitation followed by high-throughput sequencing (CLIP-Seq) captures short RNA fragments crosslinked to a specific RBP, here AGO2. To try and pinpoint sequence patterns (motifs) that drive RBP recognition (what sequence does AGO2 bind to?), we attempt to use computational analysis. Deep convolutional neural networks (CNNs) have proven useful for this task: their filters can capture sequence motifs and higher-order dependencies.

In the following exercises you will:

- Preprocess the data: convert raw CLIP-Seq reads into one-hot encoded sequences suitable for CNN input.
- Build and train a CNN model in PyTorch to predict the binding probability of a given sequence to the AGO2 protein.

$$f(\text{sequence}) = P(\text{binding}|\text{sequence})$$

  - For example: $f(\text{ACGUACGUACGU}) = 0.87$ means that the model predicts a probability of 0.87 that the sequence `ACGUACGUACGU` binds to AGO2.
- Tune hyper-parameters and benchmark performance
- Visualize binding motifs

2.1.1.1 Dataset & background

You can download the dataset files from here or from kaggle

File	Purpose
`CLIPSEQ_AGO2.train.positives.fa`	Training positives
`CLIPSEQ_AGO2.train.negatives.fa`	Training negatives
`CLIPSEQ_AGO2.ls.positives.fa`	Validation positives
`CLIPSEQ_AGO2.ls.negatives.fa`	Validation negatives

Unzip the archive and place the four `.fa` files in the same directory as the notebook, keeping these exact names. On Google Colab, upload them into the session (they are deleted when the session restarts). On Kaggle, add the linked dataset to your notebook. The archive also contains a `utils/` folder - you do not need it, `task-1.ipynb` is self-contained.

References:

Pan, Xiaoyong, and Hong-Bin Shen. "Predicting RNA-protein binding sites and motifs through combining local and global deep convolutional neural networks." Bioinformatics (Oxford, England) vol. 34,20 (2018): 3427-3436. https://doi.org/10.1093/bioinformatics/bty364

2.1.1.2 Compute for this task

This is the heavier of the two tasks: about 92,000 training sequences (plus 1,000 for validation), and you will train several configurations. Use a GPU (see Section 1.3), and debug on a few hundred sequences for a single epoch before you scale up.

2.1.2 Instructions

A. Read the Introduction and Materials and methods (up to and including 2.4) of the publication above (Pan & Shen, 2018). Their work is our inspiration rather than a specification:

we build one global CNN of our own design and skip their local CNN and the ensemble. Section 2.4 is the basis for the motif part of this task.

Q1 Explain the role of the AGO2 protein.

Q2 How can CNN filters capture sequence motifs? What is the purpose of the convolutional layers, and how do they differ from fully connected ones? Once you have built your model, support your answer with its own numbers: how many parameters are in your first convolutional layer, and how many would a fully connected layer covering the same input need?

Q3 Report number of positive and negative sequences in the train and validation sets.

Note on splits: this task has only two sets. Train only on the `train.*` files. The `ls.*` files are never trained on, and they play both roles that HW1 and HW2 split between a validation and a test set: you compare architectures by their `ls.*` score, and you also report your final numbers on `ls.*`. There is no third set here. Because you chose your architecture by looking at these sequences, your reported score is a mildly optimistic estimate of true performance, which is worth a sentence in your report.

B. Implement the CNN model in `task-1.ipynb` (Section 1, `TODO-1`) and the validation loop (Section 2, `TODO-2`).

One fact about the shapes is worth knowing before you start, because it is about the data rather than about PyTorch. Your input is $(N, 1, 4, L)$, where the height is the four nucleotide rows. The first convolution spans all four of them at once, so after it the height is 1 and stays 1: from there on, every layer is working along the length of the sequence only.

C. Train the model using different configurations and any hyper-parameters you find rel-

evant. Remember, you are limited to the GPU time quota. A configuration here means the model layers and their settings. For example: number of convolutional layers, their kernel sizes, and the number of filters in each layer. You can also experiment with different learning rates, batch sizes, and optimizers.

Q4 Plot the learning curves on the train set (loss vs epochs), and theorize how your architecture affected the learning process.

D. Model evaluation: In HW1 we have seen how TPR and FPR can be used to measure the performance of a binary classifier. In this task, we will use the ROC curve and AUC to evaluate our model. Here, we train a model that outputs a **logit**, a raw score which the sigmoid turns into a probability (higher = more likely bound). To turn probabilities into 0/1 calls we pick a threshold. Different users may want high sensitivity or high specificity and may choose different thresholds accordingly.

The Receiver Operating Characteristic (ROC) curve shows how sensitivity (True Positive Rate, TPR) trades off against 1-specificity (False Positive Rate, FPR) as you vary the decision threshold from 1.0 down to 0.0.

Reminder: Each threshold classifies examples based on their probability.

$$\tilde{y}_{i,\text{binary}} = \begin{cases} 1 & \text{if score} \geq \text{threshold} \\ 0 & \text{otherwise.} \end{cases}$$

Because this threshold can be varied, we are interested in how the model performs across all thresholds, not just the one that maximizes accuracy. For each threshold, we can compute:

- $\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$ "of all real positives, how many did we catch?"

- FPR = FP/(FP + TN) "of all real negatives, how many did we wrongly call positive?"

A worked example. Say we have 10 validation sequences, 5 that really bind (1) and 5 that do not (0), and the model gives each one a probability:

probability:	0.95	0.88	0.71	0.63	0.55	0.44	0.31	0.22	0.14	0.05
true label	1	1	0	1	1	0	1	0	0	0

Now sweep the threshold. Everything at or above it is called "binds", everything below is called "does not bind":

threshold	TP	FP	FN	TN	TPR	FPR
0.90 (very strict)	1	0	4	5	0.2	0.0
0.60	3	1	2	4	0.6	0.2
0.50	4	1	1	4	0.8	0.2
0.20 (very permissive)	5	3	0	2	1.0	0.6

Read the pattern: lowering the threshold can only raise TPR and FPR together, never one alone. A strict threshold misses real binders (low TPR) but almost never raises a false alarm (low FPR). A permissive one catches every binder (TPR 1.0) at the cost of calling non-binders positive (FPR 0.6). Each row of that table is one point (FPR, TPR) on the ROC curve, and the curve is what you get by sweeping the threshold continuously.

How to read the curve. The x-axis is FPR (lower is better) and the y-axis is TPR (higher is better), so the ideal model hugs the top-left corner: it catches every real binder

without a single false alarm. The diagonal from $(0, 0)$ to $(1, 1)$ is what random guessing looks like, so a curve sitting on the diagonal means the scores carry no information. The further your curve bows towards the top-left, the better the model.

This gives us two things:

- **Comparing models.** The Area Under the ROC Curve (AUC-ROC) is the area under that graph, a single number summarising performance across *all* thresholds. 0.5 is random guessing, 1.0 is perfect. Because it does not depend on any one threshold, it is a fair way to compare two configurations.
- **Choosing a threshold.** The curve shows what each choice costs. A screening assay where missing a binder is expensive would pick a point high up the curve and accept the false alarms. An experiment where testing each candidate is expensive would pick a point far to the left and accept missing some binders.

To calculate the AUC-ROC and plot the ROC curve, you can use the `roc_curve` and `roc_auc_score` functions from the scikit-learn library.

Q5 Final held-out metrics: for your best model, report the accuracy, plot the ROC curve, and report the AUC - all on the validation (`ls.*`) set.

Q6 Report the model's configuration: a short layer-by-layer description (shapes and total trainable parameters).

2.1.3 Interpretation: Motif Discovery

Now that we have a model, we can predict for each sequence whether it binds or not, but we also want to know *why* it made that decision. That means, we would like to find out what sequence patterns the model learned to recognize as binding sites. This is known as *motif discovery*. Sequence motifs are short, recurring nucleotide patterns that are statistically enriched (more frequent) in binding sites and we will use them to capture the base-level preferences of an RBP. A motif can be represented as a Position Weight Matrix (PWM) - see example below: for each position in the motif (say length 10), we sum how often A, C, G, or U occurs across many aligned instances and convert those counts to probabilities. Tall columns in a logo mean strong base preference, flat columns mean weak ones.

	1	2	3	4
seq 1	A	C	G	U
seq 2	A	U	G	U
seq 3	A	C	U	U

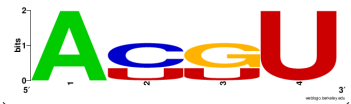
Raw Sequences

$\Longrightarrow$

	A	C	G	U
1	1	0	0	0
2	0	$\frac{2}{3}$	0	$\frac{1}{3}$
3	0	0	$\frac{2}{3}$	$\frac{1}{3}$
4	0	0	0	1

PWM

$\Longrightarrow$



Sequence logo (motif)

In full research pipelines, investigators locate motifs by opening up a trained convolutional neural network and inspecting its first-layer filters, which act like motif detectors. That is exactly what we will do here, and the procedure is short enough to write yourself.

The idea is that you do not need to read a filter's weights to find out what it looks for. You can watch it work. We will apply the first convolutional layer on positive (binding) examples from our dataset, and record the outputs. Wherever a filter produces a high value, it has met a pattern it "likes". We can then save all preferred patterns. We will use

those to create a PWM, and the PWM drawn as letters is the logo.

"Likes" needs a number attached to it. We use the same rule as the paper: for each filter we compute a **cutoff**, and any position where that filter's output is above its cutoff counts as a hit.

$$\text{cutoff} \;=\; \text{mean} \;+\; 0.5 \times (\max - \text{mean})$$

The mean and the maximum are taken over that filter's output across all sequences and all positions, so the cutoff sits halfway between a filter's typical response and its strongest one, and each filter gets its own. There is nothing here for you to tune.

A worked example. Take one filter of width $k = 4$. It sits on the sequence, reads the 4 bases under it, and returns one number. Then it slides along by one and does it again:

	G G A C G U A A		
pos 1	<u>G G A C</u> G U A A	→	0.0
pos 2	G <u>G A C G</u> U A A	→	0.1
pos 3	G G <u>A C G U</u> A A	→	**2.4**
pos 4	G G A <u>C G U A</u> A	→	0.3
pos 5	G G A C <u>G U A A</u>	→	0.0

Five windows, so five numbers. Do the same for two more sequences:

	sequence	pos 1	pos 2	pos 3	pos 4	pos 5
seq 1	`GGACGUAA`	0.0	0.1	**2.4**	0.3	0.0
seq 2	`CCUUAGCA`	0.2	0.0	0.1	0.0	0.1
seq 3	`AUCGUACC`	0.0	**1.9**	0.4	0.0	0.0

There are 15 numbers in total. Their mean is $5.5/15 = 0.37$ and the largest is 2.4, so

$$\text{cutoff} = 0.37 + 0.5 \times (2.4 - 0.37) = 1.38.$$

Two of the 15 windows are above 1.38: position 3 of sequence 1, and position 2 of sequence 3. Everything else is background. For each hit we take the $k = 4$ bases starting at that position:

- seq 1, position 3 $\rightarrow$ `ACGU`
- seq 3, position 2 $\rightarrow$ `UCGU`

Those two strings are what we keep. Note they are **aligned**: both start where the filter fired, so their first letters correspond to each other, and we can stack them and count. With 4-mers from thousands of real sequences instead of two, counting the bases at each of the four positions gives the PWM, and the PWM drawn as letters is the logo.

Your own sequences vary in length (most are 200-375 bases) and are padded out to $L = 501$ before they reach the network, so with a width-10 filter each one gives $501 - 10 + 1 = 492$ numbers. Windows falling in the padding are discarded, and a filter still usually fires on a few thousand real positions across the dataset.

Each filter gives you one logo. Some will show a clear preference and others will look almost flat, which is itself informative: not every filter learns something worth reading.

In short: apply the filter, keep the windows that score above the cutoff, count them into a PWM, draw the logo.

E. **Read a motif off the model.** Complete the readout in `task-1.ipynb` (Section 3.1, `TODO-3`) by assembling the provided helpers. Export a handful of filters, turn them into logos with WebLogo, and include two or three in your report. Expect some to come out flat: a filter that learned nothing is a result too.

Q7 Looking at your logos: what kind of sequence does your model prefer? Which filter did you take forward to the next step, and why? What is it about its picture that made you pick it? Any outliers?

As we have seen in class, learned filters (weights) differ from model to model and are not always easily explainable. Your logos will not match your classmates' exactly, and that is expected. Usually these results are validated against known literature.

F. **Test the motif.** A logo is a *claim* about what the model responds to, and so far nothing has tested it. In `task-1.ipynb` (Section 3.2, `TODO-4`) you will put the claim to the model directly: score a batch of random sequences, then plant your motif into those same sequences and score them again, using several copy numbers so you get a **dose-response** curve. If the model really responds to the pattern, more copies should push the score up.

The part that makes this an experiment rather than a demonstration is the **control**. A rise in score after planting a G/C-rich motif could mean the model responds to that specific pattern, or merely that it likes G/C-rich sequences. You have to build a comparison that separates those two explanations, run it alongside the real motif, and say in your report what you chose and why it isolates what you want.

Q8 Report your dose-response curve for the real motif and for your control, and state what

control you used and why it isolates the pattern from its base composition. Does the model respond to the pattern, to its composition, or to neither?

2.1.4 Theoretical question

Q9 Can the fully connected layers in the model be of any size? Why or why not? What is the effect of the size of the fully connected layer on the model's performance?

2.2 Task 2 - Transformer for nanopore basecalling

2.2.1 Introduction

Nanopore sequencing reads DNA by pulling a single strand through a protein pore and measuring the ionic current across it. The bases inside the pore set the current, so a read arrives as a noisy one-dimensional signal called a *squiggle*:

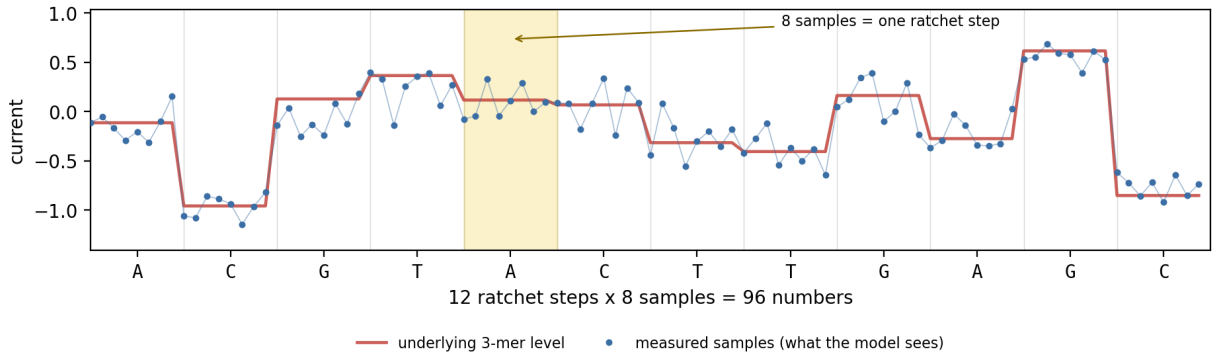


Figure 1: A squiggle from a 12-base strand. The red line is the current each 3-mer in the pore produces. The blue dots are the individual measurements the model receives, eight for every step of the ratchet.

Basecalling is the inverse problem: recover the sequence from the squiggle. In Figure 1 you can read the answer off the picture because the bases are printed underneath. Production basecallers, such as Oxford Nanopore's Dorado, now do this with a Transformer. The model gets only the blue dots, generated by sequencing, and attempts to reconstruct the sequence. In other words, it learns (approximates) a function that takes a list of current measurements and returns a string of bases:

$$f(\underbrace{\text{squiggle}}_{\text{96 numbers, above}}) = f(s_1, \ldots, s_{96}) = \underbrace{\texttt{ACGTACTTGAGC}}_{\text{12 bases}}$$

2.2.1.1 Compute

Remember to use a GPU for this task (see Section 1.3), as the computations are intensive. Decide what each run is testing before you launch it, and control the number of training steps.

2.2.2 The data

You will generate your reads using a simulation, instead of being provided with a dataset.

In Task 1 you were given files of RNA sequences, with labels that were already established in a lab. Here you will create a random strand of DNA, and simulate the current a pore would measure as that strand passed through it to assign a squiggle to it. These simulated squiggles are your input ($\mathbf{X}$), and the DNA sequence of the strand is your label ($\mathbf{y}$). Because you created the DNA strand in the first place you already know every base of the answer. The pore's physics is understood well enough that simulators like ours are a standard tool, used to benchmark basecallers and to test pipelines where no real data exists. Production basecallers are trained on real reads instead, labelled by aligning them to a genome that is already known.

A batch is generated at every training step, so your model never sees the same read twice and there is nothing to hold out: no train, validation and test split. `evaluate` draws its reads from a fixed seed, so any two models you train are scored on exactly the same reads, as is every submission at grading time.

Two settings control the noise in the reads, and you will use them to test how your model performs under different conditions:

- `noise`, the strength of the noise on the current, from `0` (clean) to about `1` (noise as strong as the signal itself).
- `uneven`, how much the noise level varies from base to base along the read, from `0`

(the same everywhere) to `1`.

2.2.2.1 The pore simulation

Three bases sit in the pore at once and all three affect the current. Every possible 3-mer has its own characteristic current level, and that table of $4^3 = 64$ numbers is the pore model. A few examples from ours:

```
AAA  +0.30        ACG  +0.13        GTA  +0.12
AAC  -1.04        CGT  +0.37        TAC  +0.07
```

The strand ratchets through one base at a time, so the pore holds a different 3-mer at every step. Reading the strand `ACGTAC`, the pore sees a sliding window:

```
strand       A C G T A C


pore holds   A C G              ->  +0.13
               C G T            ->  +0.37
                 G T A          ->  +0.12
                   T A C        ->  +0.07
```

One step gives one level. A sliding window of 3 over 6 bases only fits four times, and our simulator wraps the first two windows around the end of the strand, treating it as circular, so that a 60-base read gives exactly 60 levels. This is also why a base cannot be identified on its own: the level at each step depends on the bases around it as well. Each level is held for 8 samples (`DWELL` in the notebook) while the strand pauses there, and Gaussian noise is added on top. A read therefore arrives as $60 \times 8 = 480$ numbers.

That is exactly Figure 1: the red line is those levels, the blue dots are the same thing

measured with noise, and each flat stretch is one step of the ratchet.

2.2.2.2 The two strands, and the adapter

DNA is double-stranded and antiparallel. The pore reads whichever strand it happens to capture, 5' to 3'. Capture the forward strand and the pore reads the forward sequence. Capture its partner and the pore reads the *reverse complement*: every base swapped for its pair (A-T, C-G), in the opposite order.

In our case, we always want the forward strand reported, regardless of which one was in the pore. That is the function f we are trying to learn:

$$f(\text{squiggle from } \texttt{ACGTAC}) = \texttt{ACGTAC} \qquad f(\text{squiggle from } \texttt{GTACGT}) = \texttt{ACGTAC}$$

Those are two different squiggles, built from different 3-mers, and f has to return the same answer for both.

So on a reverse capture the first base of the answer is set by current measured at the *end* of the read, and the last base by current measured at the start.

This split is how real sequencing works. A basecaller writes down whatever passed through the pore, and the *aligner* decides afterwards which strand it came from and flips it if needed. Here a single model does both jobs.

The problem. Two random strands look the same. Nothing in the current levels says which orientation the pore captured, and without that the model cannot know whether to reverse what it read. Real reads carry a sequencing **adapter**, a known stretch ligated on before sequencing that passes through the pore ahead of the DNA. In our generator, reverse captures carry a leftover adapter signal at the very start of the read and forward

captures do not.

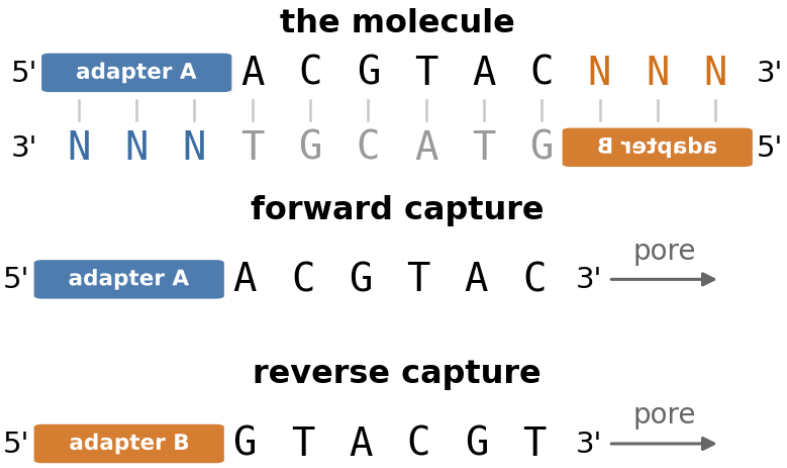


Figure 2: Each strand carries its own adapter, ligated to its 5' end, and the two adapters are different. `N` stands for the adapter's own bases, paired like the rest of the molecule. The pore pulls a strand in 5' end first, so whichever strand is captured, that strand's adapter goes through ahead of the DNA. The partner's 5' end is the right-hand end of the duplex, which is why it enters the pore as `GTACGT` and why its label is printed mirrored in the top panel.

2.2.3 From signal to tokens

The model turns one read into one sequence.

The input. One read is 480 current samples. Figure 1 is the same picture for a shorter, 12-base strand. Write the read s:

$$s = [s_1, s_2, \ldots, s_{480}] \in \mathbb{R}^{480}$$

Tokenization. There are 60 ratchet steps and 8 samples belong to each, so the read is cut into 60 chunks of 8. Each chunk is a **token**: the unit the model reasons about, the way a

word is the unit in a language model.

Embedding. A token is 8 raw numbers, which is not something attention can work with. The **embedding** turns each token into a vector of width d:

$$\underbrace{[\,s_1, \ldots, s_8\,]}_{\text{token 1}} \longrightarrow x_1 \in \mathbb{R}^d \qquad \underbrace{[\,s_9, \ldots, s_{16}\,]}_{\text{token 2}} \longrightarrow x_2 \in \mathbb{R}^d$$

Keeping the two apart matters, because Tutorial 4 did the second one differently. There a token was a word and the embedding was a lookup in a learned table, one row per word, which works because there are finitely many words. A token here is 8 real numbers, so there is no table to look up: a convolution of width 8 and stride 8 computes the vector instead, the Task 1 trick again, stepping one ratchet step at a time. Same role in the pipeline, different mechanism. The notebook's `Tokenizer` layer does both steps.

We do that for all 60 tokens and stack the results as rows. That matrix is the model's representation of the read:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{60} \end{bmatrix} \in \mathbb{R}^{60 \times d}$$

d is yours to choose and is called `d_model`. From here on we follow the usual shorthand and call the vector x_i "token i" as well.

2.2.4 Self-attention

Each token x_i produces a query, a key and a value, and the attention weight from token i to token j compares i's query with j's key, scaled by the key width d_k:

$$a_{ij} = \mathrm{softmax}_j\left(\frac{q_i \cdot k_j}{\sqrt{d_k}}\right), \qquad z_i = \sum_{j=1}^{60} a_{ij}\, v_j.$$

The sum runs over every token j, so any token can draw on any other, near or far.

2.2.5 Reading attention maps

The attention weights are the one part of a trained model you can look at directly. For a single read, a_{ij} is a number for every pair of positions, so one head gives one square grid: row i is the position being computed, column j is the position it looked at, and brightness is the weight. Each row is a softmax, so it sums to 1. An even split over all 60 positions would be $1/60 = 0.017$ everywhere, and a bright cell means a single position took a large share of that row.

Three shapes are worth being able to recognise by sight:

2.2.6 Building the model

The model is a list of blocks applied in order:

$$\underbrace{\text{squiggle}}_{\text{input}} \longrightarrow [\text{Tokenizer}] \longrightarrow +[\text{Positional encoding}]$$

$$\longrightarrow n_{\text{blocks}} \times [\text{Encoder block}] \longrightarrow [\text{Output head}] \longrightarrow \underbrace{\text{sequence}}_{\text{output}}$$

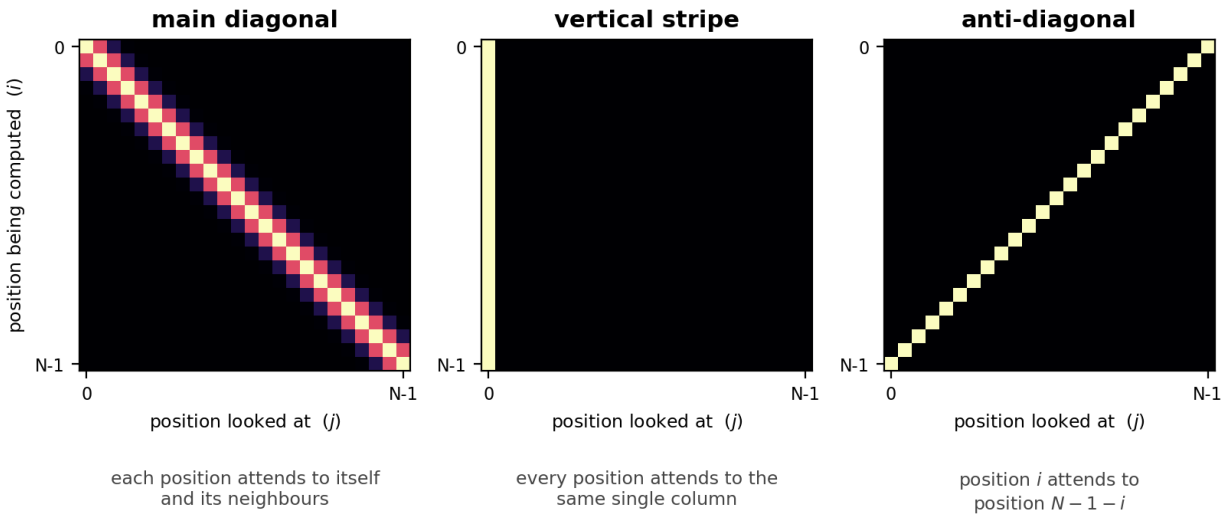


Figure 3: The three shapes, drawn by hand rather than taken from a model. A trained head may show one of them, a mixture of them, or nothing recognisable at all.

block	what it does
Tokenizer	the convolution above: turns the 480 samples into 60 tokens.
Positional encoding	*adds* a fixed position signature to each token rather than replacing it, so a token keeps its width. Attention on its own has no notion of order.
Encoder block	self-attention, then a small per-token network, plus several things that keep a deep stack trainable. You stack as many of them as you choose, one feeding into the next.
Output head	turns each token into its 4 scores, like the `Linear` layer that ended your CNN in Task 1, with 4 outputs instead of 1.

Section 3 of the notebook has the code for all of them, and explains the settings each one takes: how wide a token is, how many encoder blocks you stack, and so on.

2.2.7 Model output and loss

Model output. After the encoder blocks, a `Linear` layer maps each token to 4 numbers, one per base. Stacked, they form a matrix we call O, laid out like the one-hot encoding of Task 1:

$$O \in \mathbb{R}^{60\times 4}$$

Row i holds the four scores for the i-th base of the answer. These are raw scores, not probabilities: higher means more confident. A `softmax` over a row turns it into four probabilities that sum to 1, and the largest one is the base we call.

	A	C	G	T	
scores, base 1	2.7	−0.4	0.1	−1.2	
probabilities	0.88	0.04	0.06	0.02	⇒ A

Reading the winner off all 60 tokens gives the called sequence $\hat{y}$, which is what we compare against the true sequence y:

$$\hat{y} = \texttt{A G} \ldots \qquad \text{against} \qquad y = \texttt{A C} \ldots$$

Loss. Accuracy compares $\hat{y}$ with y, but you cannot train on it, because taking only the largest entry throws away how confident the model was and gives no gradient. The loss instead uses O directly: at each of the 60 positions it takes the probability the model assigned to the *correct* base and penalises it for being small. That is **cross-entropy**, one value per position, averaged over the 60 positions and over the batch.

Task 1 did the two-choice version of exactly this. Here, the softmax is applied inside the loss, so your model's output head must output the raw scores O with no softmax after it.

2.2.8 Instructions

Work through `task-2.ipynb` in order. Each part corresponds to one section of the notebook.

A. **Look at the data** (Section 1). Plot a few reads (forward and reverse) and their labels. Look at the adapter signal and see how it separates the two orientations.

Q10 What separates a forward capture from a reverse one in the signal? And on a reverse capture, which part of the signal determines the first base you have to report?

B. **Run the CNN** (Section 2).

Q11 Report your CNN's accuracy overall and on each orientation, and explain the result. What property of a convolutional architecture is responsible, and would adding more filters (`C`) help?

C. **Train the Transformer** (Section 3, `TODO-1`). Choose your sizes, and train it on the same data.

Q12 Report your Transformer's accuracy next to your CNN's. What changed, and what does that tell you about the two architectures?

D. **Choose a configuration** (Section 4, `TODO-2`). Vary the model sizes and the data settings. Save your best model at the end of Section 4 and include the resulting `basecaller.pt` in your submission. That file will be used for grading and will be tested under different noise conditions.

Q13 Report your final configuration (`d_model`, `n_heads`, `n_blocks`, `d_ff`, and total parameters) and justify it with your plots, including accuracy against noise for at least two model sizes.

E. **Ablation** (Section 5, `TODO-3`). An *ablation* removes one part of a working model and retrains without it, so that whatever performance is lost can be attributed to that part. Here the part is the positional encoding: rebuild with `use_pe=False`, keeping every other size fixed, and retrain.

Q14 Report both orientations with and without the positional encoding. Account for what changed, and for any difference between the two orientations.

F. **Attention Maps** (Section 6, `TODO-4`). Plot the attention maps on a reverse read to see which attention heads are active. Then silence each head in turn and re-score, so you can compare what the maps suggest against what the model actually learnd. Whether attention weights count as an explanation at all is an open argument in the literature.

Q15 Is it possible/impossible to identify specific attention heads that are specifically responsible for the orientation? How can you tell? What do the attention maps show, and how does that relate to your answer in *Q11*?

2.2.9 Theoretical questions

Q16 In a short paragraph, relate self-attention here to its use in protein language models, which attend across amino-acid residues. Why is the ability to look across a whole sequence useful in biology?

Q17 The embedding is a convolution of width 8 and stride 8. Explain what each of the two numbers is doing, and why both of them are 8 here. If you changed the stride to 4 and left

everything else alone, what would be the effect?

3 Submission Guidelines

Submit a single ZIP file with the following structure:

```
final_project_ID1_ID2.zip
  src/                 # your code files
     task-1/task-1.ipynb   # Task 1: CNN for RNA-protein binding sites
     task-2/task-2.ipynb   # Task 2: Transformer for nanopore basecalling
     task-2/basecaller.pt  # Task 2: the trained model you want graded
     any other code files for reproducibility...
  llm_chats/           # exported LLM conversations (see note below)
  report_ID1_ID2.pdf   # < 5 pages + charts
```

Do not include any data files in the submission, as they are very large and already available for download. `basecaller.pt` is not a data file - it is a few megabytes of model weights, and it must be there.

LLM chat logs. As in HW1 and HW2: if you used LLMs (e.g. ChatGPT, Claude, Gemini, Copilot) at any stage, export the full conversation(s) into `llm_chats/` (PDF, Markdown, or a `.txt` with shareable links).

3.1 Written report guidelines

The report should be a concise summary of your work. It should show your understanding of the model, results and reasoning behind your decisions. No need to include code, any equations or algorithms used should be mentioned if we saw them in class or explained otherwise.

- **Content**:

  - Include your names and IDs on the first page
  - Include answers and results of all parts in the report.
  - Include all plots and figures that help explain your results.
  - Use clear markers for each task and question.

- **Length**: Maximum 5 pages (excluding charts, figures and tables; equations count towards length).
- **Font**: Readable (e.g., Arial) with a minimum size of 11pt and standard margins.
- **Explanations**: Provide clear, short explanations for your model choices, esp. if they diverge from what we've seen in class. Include any challenges you faced and how you solved them.
- **Charts**: Include relevant figures, make sure they are clear and labeled, and provide a brief explanation of how they relate to your results.

3.2 Grading rubric

Component	Pts
Written Report	30
Full implementation of the models	30
Theoretical Questions	20
Model performance	15
Results reproducibility	5

3.3 Academic integrity & help

- Use any online docs or LLMs, do not share code or prompts across pairs.

- Post questions in the Final project Moodle forum. Emails regarding the project will not be answered. For private issues email the TA.

Good luck!

3.4 FAQ

Q: Can I use Kaggle/Colab? A: You may use any platform for development, but your final submission should follow the required folder structure and include files as specified.

Q: What libraries can I use? A: You may use any library, but you cannot use any pre-built models - in particular, assemble the Transformer from the provided blocks rather than using `torch.nn.Transformer`.

Q: Do I need a GPU to run this assignment? A: Using a GPU will significantly speed up training times for both tasks. You should use the free GPUs available on platforms like Google Colab or Kaggle.

Q: What language may I use for the report and code? A: Report: English recommended, Hebrew accepted. Code: any language, but Python is recommended for compatibility with the provided code.

Q: What should I do if my results are different each run? A: Make sure to set all random seeds (e.g., random_state=42 in train_test_split and for any random number generators you use) for reproducibility.

Q: What Loss function should I use? A: You can use any loss function that is suitable. If we have not seen it in class, please explain it in your report.

Q: What does code reproducibility mean? A: Code reproducibility means that anyone who

runs your code and provides the data should be able to obtain the same results as you did.